

The Sugar Vault

An Arduino controlled candy & chocolate dispensing machine

CODE BREAKDOWN

Start Date: 5/18/2026

End Date: 6/28/2026

Software Used: Google Docs, SOLIDWORKS, draw.io, Arduino IDE, GitHub, Bambu Studio

Project GitHub: <https://github.com/cchapman9x/the-sugar-vault>

Project Creator:

- Caleb Chapman | Montgomery College | Electrical Engineering

```
const int dirPin = A2;
const int stePin = A3;
const int buttonPin = 3;
const int ledPin = 2;

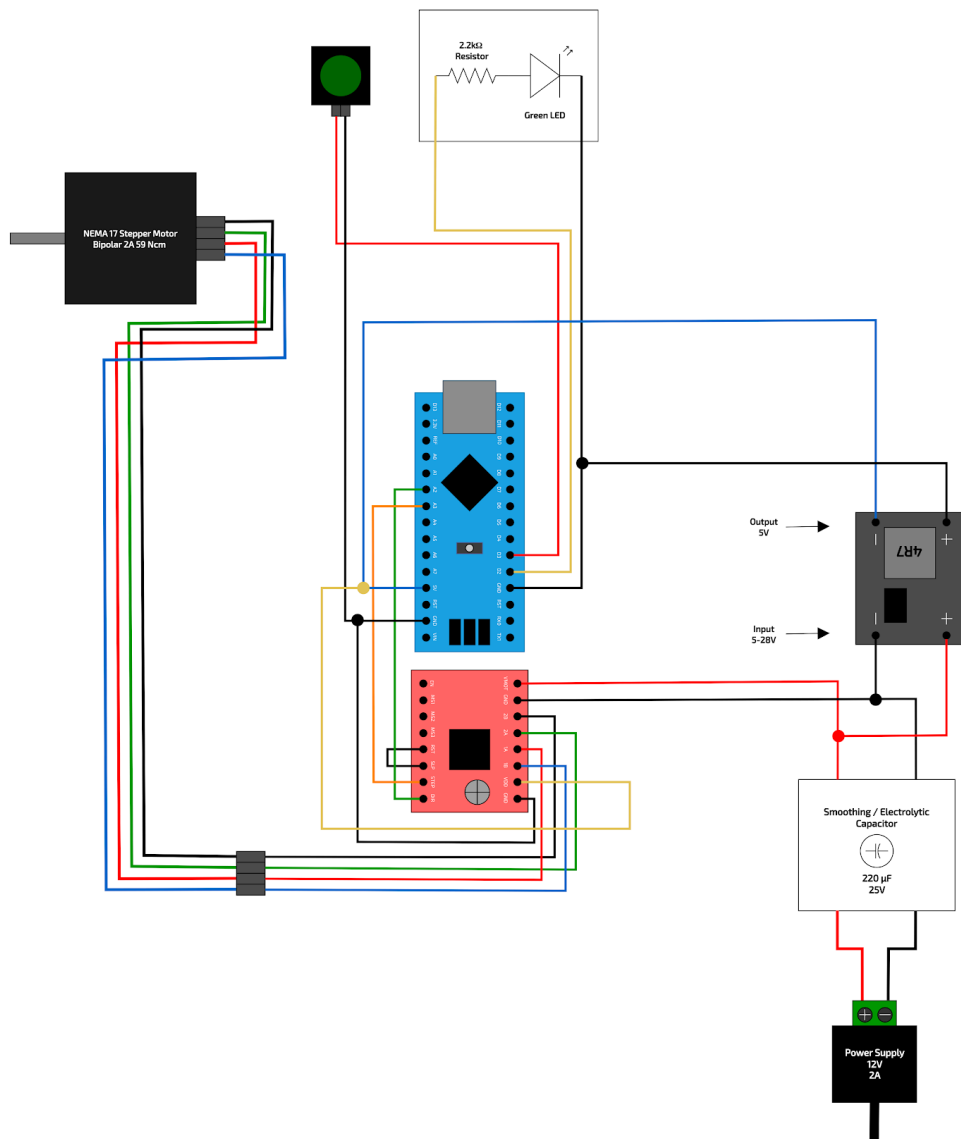
void setup() {
  pinMode(dirPin, OUTPUT);
  pinMode(stePin, OUTPUT);
  pinMode(buttonPin, INPUT_PULLUP);
  pinMode(ledPin, OUTPUT);

  digitalWrite(dirPin, LOW);
  digitalWrite(ledPin, HIGH);
  digitalWrite(ledPin, LOW);
}

void loop() {
  bool dispenseCandy = digitalRead(buttonPin) == LOW;
  if (dispenseCandy) {
    digitalWrite(ledPin, HIGH);
    for (int i=0; i < 200; i++) { // this is 200 steps
      digitalWrite(stePin, HIGH);
      delayMicroseconds(7000);
      digitalWrite(stePin, LOW);
      delayMicroseconds(7000);
    }

    digitalWrite(ledPin, LOW);
  }
}
```

The Sugar Vault Electronics Schematic



By Caleb Chapman

Code Breakdown

Pin Initialization

```
const int dirPin = A2;  
const int stePin = A3;  
const int buttonPin = 3;  
const int ledPin = 2;
```

At the top of the code, I initialized 4 variables, each representing a pin. Please note that this code was originally made using a breadboard setup, so some wire colors may be different from what is shown on the final schematic.

- **dirPin** - The dirPin is set to pin A2 on the arduino. Pin A2 is connected to the DIR pin of the A4988 stepper motor driver. It should be noted that pin A2 is an analog pin but can also be used as a digital pin. The arduino sends either a HIGH or LOW digital output signal to this pin which the A4988 then receives. The signal tells the A4988 driver which direction to rotate the stepper motor only when STEP pulses are sent. The A4988 changes the direction of the motor by energizing the motor coils in different orders. This allows the driver to control both CW and CCW movement of the NEMA 17 motor.
- **stePin** - The stePin is set to pin A3 on the arduino. Pin A3 is connected to the STEP pin of the A4988 stepper motor driver. The arduino also sends HIGH and LOW digital output signals to this pin which the A4988 receives. Though the DIR pin controls the direction of rotation of the stepper motor, the STEP pin is what controls / triggers the movement altogether. Without the STEP signal, the DIR signal can only set the direction but the motor won't physically move. This can be further explained with a **timing trace**! So each pulse of the STEP pin tells the A4988 to move the motor one step in the direction given by the DIR pin signal.
- **buttonPin** - The buttonPin is set to pin 3, which is pin D3 on the arduino. Pin D3 is connected to the red / signal wire of the green momentary push button which acts as the user input in this system. The buttonPin is set to INPUT_PULLUP in the code which means the arduino will use its pull-up resistor. When we have a button connected to an Arduino, it needs a clear voltage state or digital state, either HIGH or LOW, in order to be read correctly. If the button input is not connected to a clear state, it can "float," which means the Arduino may read random HIGH or LOW values because the pin is picking up electrical noise instead of a clear digital signal. INPUT_PULLUP fixes this by turning on a small internal resistor inside the Arduino that weakly connects the input pin to 5V. This means that when the

	button is not pressed, the input pin reads HIGH. In this project, D3 is internally connected to 5V through the pull-up resistor because of the INPUT_PULLUP setting in the code. When the button is pressed, the button pulls D3 down to GND, so the Arduino reads LOW. This is why the code checks if the button reads LOW in order to trigger the motor. When the button is pressed, it pulls pin D3 LOW, creating the LOW digital input state that the arduino reads. This is how the arduino knows to send the digital output signals to pin A3 which are received by the A4988 driver which then triggers the motor. Note also that the D in D3 means that this pin is for digital signals which is why after learning more about the formatting of the arduino, I connected it here. ○ Not Pressed = HIGH ○ Pressed = LOW ● ledPin - The ledPin is set to pin 2, which is pin D2 on the Arduino. Physically, this pin connects to a 2.2kΩ resistor, which then connects to the actual LED. The resistor limits the amount of current flowing through the LED so the LED and Arduino pin are protected. When the button is pressed, the LED turns on, indicating that the dispensing wheel is turning and dispensing candy. It turns off once the wheel has completed its programmed rotation. The LED is an output device because the Arduino sends a digital signal to control whether it is on or off.
The setup() Function	In Arduino programming, the setup() function is used as the one-time startup section of the program. It runs once when the Arduino first turns on, resets, or starts the sketch, which is the Arduino program. It is basically the function where you can set default values, set pin modes, and initialize libraries. This is the <u>preparation</u> function for the program. <pre>pinMode(pin, MODE);</pre> ● pinMode() - The pinMode() function is used for setting the modes of several pins. It allows us to control the behavior of each pin and determine if we are inputting data with the pin or outputting it. ○ dirPin & stePin - These pins are set to OUTPUT mode because the arduino uses them to output digital signals to the A4988 motor driver. ○ buttonPin - The buttonPin is an input pin because it receives input from the user to activate the motor. However, the button also

	needs a clear default state so the pin does not float. Because of this, the pin mode is set to INPUT_PULLUP instead of just INPUT. This allows the Arduino to use its internal pull-up resistor, making the button read HIGH when it is not pressed and LOW when it is pressed. ○ ledPin - This pin is set to OUTPUT because it is used to convey that the dispensing wheel is on to the user. ● digitalWrite() - This function is used to control the digital output signal of the output pins. It will tell the pins to go either HIGH or LOW. When an output pin is set to HIGH, the Arduino sends voltage out of that pin. When an output pin is set to LOW, the Arduino brings that pin down to 0V/GND. In this project, digitalWrite() is used to set the direction signal for the motor driver and to turn the LED on or off. ○ dirPin - We set the dirPin to LOW because the digital output signal controls the direction of rotation. A LOW signal means that the motor will turn CCW while a HIGH signal means the motor will turn CW. I wanted the motor to turn CCW. ○ ledPin - HIGH turns the LED on and LOW turns it off. So we use this function simply to control the LED.
The loop() Function	In arduino programming, the loop() function is used as the main repeating part of the program. After the setup() function runs once, the loop() function that runs on repeat as long as the arduino is powered. The loop in the program does the following: 1. Read the value of the button and check for a LOW signal 2. If the button is pressed, meaning the digital signal is LOW, do the following: a. Turn the LED on b. Make the motor do one full 360° turn c. Once the motor is finished turning, turn off the LED 3. Repeat Lets look inside the loop() function ● dispenseCandy - This is a Boolean variable that stores either true or false. It gets its value from the expression digitalRead(buttonPin) == LOW. The digitalRead() function first reads the state of buttonPin. Since the button uses INPUT_PULLUP, pressing the button makes

	the pin read LOW. If <code>digitalRead(buttonPin)</code> returns LOW, then the comparison becomes true, meaning <code>dispenseCandy</code> is set to true and the dispensing cycle can begin because that is then plugged into the if statement. • Inside the if statement, we first turn on the LED to indicate that the dispensing wheel is moving. • We then turn the wheel 360° using a for loop. A single motor step is about 1.8°. I needed the motor to rotate 360°, so this involved a little bit of math. I divided 360 by 1.8, which gave me 200. This meant that the motor needed 200 steps in order to complete one full 360° rotation. Because I knew the exact number of iterations needed, I decided that using a for loop would be best for making the motor complete 200 steps. • I set <code>i = 0</code>, and as long as <code>i</code> was less than 200, the motor would keep stepping while the for loop kept track of the number of steps. In order to make the motor step, I used <code>digitalWrite()</code> to set the STEP pin HIGH, starting the STEP pulse. I then used <code>delayMicroseconds(7000)</code> to give the motor time between pulses, which makes the movement smoother and steadier. I wanted the motor to move slowly, so I set the delay to 7000 microseconds. This is equal to 0.007 seconds, or 7 milliseconds. I then set the STEP pin LOW to end that single step pulse and added another 7000 microsecond delay before the next pulse. The for loop repeated this process 200 times, resulting in a smooth 360° turn. • Once the for loop ends, the LED is turned off, indicating that the wheel has stopped moving.
--	--